The Triage Station: A Physically-Faithful Package-Triage Environment for Humanoid Robots in a Reconstructed Warehouse

Gym-Anything for Robotics
l3_agents@proton.me

Abstract

We present a package-triage simulation environment modeled on a real industrial workflow: a humanoid robot standing at a sortation cell must pick parcels from a bin and place them *label-up* on a moving take-away conveyor, where an overhead camera must optically decode each parcel's shipping barcode. The environment combines (i) a photorealistic neural reconstruction of a real warehouse *with its scanned collision geometry*, so the robot balances on the real building's uneven floor; (ii) a fully physical sortation cell—treadmill-driven conveyors, a gravity transfer pan, a cleated return incline, and a drop-fed steel bin—validated by instrumented audits; (iii) a diverse parcel population with real masses, dimensions and materials, including FEM-simulated deformable poly bags; (iv) a free-standing Unitree G1 humanoid balanced by the manufacturer's own pretrained locomotion policy, with no world-weld or stabilizing constraint; and (v) a cheat-resistant optical verifier whose pass signal is a real barcode decode from rendered pixels. We document the construction pipeline, governing equations and parameters, verifier design, measured validation results, and the methodology under which every subsystem was accepted: *measure before claiming; render before believing*. Task solving is deliberately left open; this is an environment release for policy builders.

1 Introduction

Contemporary humanoid demonstrations—most prominently package-triage livestreams in which a robot works a sortation cell for hours—show sustained logistics work in which success is *orientation-sensitive*: a package placed barcode-down is a failed sort regardless of how gracefully it was moved. Reproducing such a task in simulation is easy to do dishonestly: attach parcels to grippers, weld the robot to the floor, read labels from simulator metadata, and count successes with privileged state. Each shortcut produces an environment whose solutions do not transfer.

This environment was built under a single governing rule: **the simulation bends to reality, not the reverse**. Objects keep their real sizes, masses and surface properties; the robot stands because a balance controller keeps it standing; the verifier passes a parcel only if a real barcode decoder reads the label from the tunnel camera's rendered image. Where an approximation was unavoidable it is declared (§7) rather than hidden.

A second contribution is methodological. Every load-bearing claim below traces to a logged, reproducible measurement, and the project history preserves the *wrong* diagnoses alongside their corrections. Several defects reported here—a conveyor silently running at half speed in interactive sessions, a collision-floor offset that exploded robot spawns, a verifier gate that did not enforce its own documented criterion—were found only because of this discipline, and each produced a rule that is now part of the environment's regression suite.

Preprint. Under review.

2 Environment

2.1 Scene: a real warehouse, visually and physically

The scene shell is a neural reconstruction (Gaussian-splat radiance field [1], rendered natively by the simulator's NuRec pipeline) of a real warehouse interior, paired with the scan's triangle-mesh collider. Two properties follow. First, photorealism is genuine: camera views used by the verifier and available for policy observation show the real building—racks, signage, floor wear—rather than stylized CG. Second, *the floor is real*: a raycast survey on a $0.2\,\mathrm{m}$ grid measured the collision floor at 6–$14\,\mathrm{cm}$ above the nominal plane, varying across the room. The robot therefore balances on the reconstruction's true uneven surface. This was discovered the honest way: a free-standing robot spawned at nominal height had its feet inside the floor mesh and was ejected by contact depenetration; the fix (spawn height from a local raycast, §3.4) is part of the spawn doctrine.

The scene is y-up with gravity $(0, -1, 0) \cdot 9.81\,\mathrm{m\,s^{-2}}$, asserted *and read back* at the start of every run. Physics runs at $120\,\mathrm{Hz}$ (TGS solver, GPU dynamics for deformables, CCD enabled).

2.2 The sortation cell

The cell reproduces the reference station's working triangle and is built parametrically (a single builder script emits the USD stage plus a JSON config carrying every load-bearing coordinate). Table 1 lists the as-built parameters. The cell is dressed to match reference footage: two emergency stops (tray corner and tunnel post), translucent polycarbonate guards (translucency verified by a through-shot render), fastener heads, floor safety striping, and electrical conduit.

Table 1: Sortation-cell parameters (as built and measured).

Element	Parameters
Take-away conveyor	surface y=0.749 m; speed $0.35\,\mathrm{m/s}$; 11-tile kinematic ring
Scan tunnel	camera at surface $+0.52\,\mathrm{m}$; 2048^2 render; 60 k-intensity lamp
Corner transfer	gravity pan, painted steel, $\approx 18°$
Return incline	19°; 6 cm cleats; 12-tile ring
Steel bin	0.60×1.00 m floor tilted $0.58 \to 0.46$ toward the robot; walls 0.86–1.02
Deck	white work surface at the robot's front

The loop is *recirculating*: parcels discharged off the take-away cross the pan, ride the cleated incline, and are launched over the bin's far wall back to the robot—continuous arrivals without teleportation.

2.3 Parcel population

Thirteen parcel classes from a real-parcel manifest; each instance carries its true dimensions, mass, and a Code-128 shipping label (Table 2). Masses span 0.17–$4.49\,\mathrm{kg}$. Rigid parcels use cardboard friction μ_s=0.8, μ_d=0.7, restitution 0, and exact convex-hull collision. Poly bags are FEM volumetric soft bodies (hexahedral resolution 8) with Young's modulus $E = 35\,\mathrm{kPa}$ (a $20\,\mathrm{kPa}$ variant was measured to interpenetrate rigid contacts and eject bodies on recovery), Poisson ratio 0.45, density $145\,\mathrm{kg\,m^{-3}}$, and honest LDPE surface friction $\mu \approx 0.2$. Notably, a deformable's own material friction governs its contact pairs outright, so slick bags are genuinely slick.

Table 2: Parcel manifest (representative rows).

Class	Count	Example ($L \times W \times H$ cm, kg)	Body
Poly bags	5	$41.1 \times 23.2 \times 4.3$, 0.74	FEM deformable
Small boxes	3	$16.1 \times 13.4 \times 5.8$, 1.14	rigid convex hull
Medium boxes	2	$44.1 \times 24.7 \times 16.2$, 4.49	rigid convex hull
Bubble mailers	2	$34.2 \times 22.5 \times 4.2$, 0.61	rigid convex hull
Flats	1	1.8 cm thick	rigid convex hull

Labels are physically real. Every label renders at the true 4×6 in ($15\,\mathrm{cm}$) physical size regardless of parcel size—small parcels do *not* receive proportionally shrunken (sub-specification) barcodes.

Face textures are generated on aspect-correct canvases: square textures UV-stretched onto faces with aspect ratio < 0.7 were measured to squeeze bar widths below decodability. Poly-bag textures carry procedural crinkle shading in the reference palette, with the label patch composited at physical scale after shading.

2.4 The robot

A Unitree G1 humanoid (43 DoF including two 7-DoF Dex3 dexterous hands), **free-standing**: the articulation root is unconstrained. Balance and locomotion come from the manufacturer's pretrained flat-terrain policy [2] (12 leg DoF, recurrent LSTM actor, *blind*: proprioception only), the controller class deployed on physical G1s, with the published deployment gains

$$k_p = [100, 100, 100, 150, 40, 40]\times 2, \qquad k_d = [2, 2, 2, 4, 2, 2]\times 2. \tag{1}$$

The upper body (waist, arms, hands) is position-controlled independently, mirroring the real deployment architecture in which the legs policy balances while a separate controller works the arms. Arm motion physically perturbs balance and the legs visibly compensate; collisions with the station can trip the robot. This is intended.

3 Construction pipeline

3.1 Conveyance: the treadmill doctrine

Surface-velocity conveyors in the underlying engine move rigid bodies but were *measured* to move FEM deformables at exactly 0.000 m/s. Conveyance is therefore implemented as *treadmills*: rings of kinematic tiles that physically translate, carrying everything by real friction (measured: rigid 0.398 m/s and FEM bag 0.406 m/s at a 0.4 m/s command). The mechanism's rules, each paid for by a logged failure:

1. **Exact rings.** Ring span must equal $n\,\ell_{\text{tile}}$ (here ℓ_{tile}=0.3 m); anything else opens support holes at the wrap seam.
2. **Edge-wrap discharge.** Tiles carry parcels to the discharge edge (a real end-roller tip-off), then dip 0.45–0.5 m over $\approx$8 cm of travel past the edge, return underground, and teleport back while deep.
3. **The wrap teleport is a gun.** A kinematic pose jump of L=3.05 m in one $\Delta t = 1/120$ s step implies a swept velocity
$$v_{\text{implied}} = L/\Delta t \approx 366\,\text{m/s}, \tag{2}$$
and speculative/CCD contact generation fires parcels across the room even with a 5 cm clearance (caught by high-speed trace). Tiles therefore collide *only at full height* ($\Delta y > -1$ mm), and every collision toggle is followed by a physics flush—on the GPU pipeline, unflushed toggles may silently not apply.
4. **Never place furniture across a discharge.** A full-height guide fin at the belt end face-planted every arriving parcel—found only by rendering the corner, not by telemetry.

3.2 Friction pairing and gravity transfers

The engine combines pair friction by *averaging* by default, which silently manufactures grip: cardboard (0.8) on steel (0.12) averaged to 0.46, and a 14° pan held every parcel. Steel surfaces therefore declare a minimum combine mode,

$$\mu_{\text{pair}} = \min(\mu_{\text{steel}},\, \mu_{\text{parcel}}), \tag{3}$$

and a parcel slides when $\tan\theta_{\text{pan}} > \mu_{\text{pair}}$. With honest pairing the corner pan runs at $\approx 18^\circ$, and the 19° incline carries slick LDPE bags on physical cleats rather than by fictional friction.

3.3 Bin filling: exact packing and staged release

Naive grid spawns interpenetrate: a 0.20 m grid against 32–44 cm boxes ejected parcels up to 6.5 m through depenetration, *before* runtime detectors armed. The pile spawner is a first-fit-decreasing

shelf packer using **exact yaw-rotated footprints**: a parcel with footprint half-extents (h_x, h_z) at yaw θ occupies the axis-aligned bounding box

$$h'_x = |\cos\theta|\, h_x + |\sin\theta|\, h_z, \qquad h'_z = |\sin\theta|\, h_x + |\cos\theta|\, h_z, \tag{4}$$

packed with per-material clearances (+5 cm for FEM bulge). Deformable bags pack on the bin floor—an unheld bag packed high was observed to toboggan off box tops at μ=0.2—with rigid layers above, each layer based on the *real* maximum height of the layer below. Because real bins fill parcel-by-parcel, upper rigid layers spawn kinematically held and release one layer at a time onto a settled pile (belts frozen during fill; the treadmill phase state is incremental, so freezing is wrap-safe). A dry-run checker proves zero pairwise spawn overlap before any simulation.

3.4 Locomotion integration

The balance policy consumes a 47-dimensional observation assembled entirely in the *pelvis frame*, making it world-up-axis agnostic (this environment is y-up; the policy was trained z-up):

$$o_t = \big[\, \omega_b \cdot 0.25;\;\; \hat{g}_b;\;\; c \cdot (2, 2, 0.25);\;\; (q - \bar{q});\;\; \dot{q} \cdot 0.05;\;\; a_{t-1};\;\; \sin 2\pi\phi;\; \cos 2\pi\phi \,\big], \tag{5}$$

where ω_b is body angular velocity, $\hat{g}_b = R^\top \hat{g}_w$ the projected gravity, $c = (v_x, v_y, \dot{\psi})$ the velocity command, $q, \dot{q}$ the 12 leg positions and velocities about the deployment stance $\bar{q}$, a_{t-1} the previous action, and $\phi = (t \bmod T)/T$ a gait clock with $T = 0.8$ s. Actions map to leg PD targets $q^* = \bar{q} + 0.25\, a_t$ at 50 Hz (additionally validated at 60 Hz).

Two integration rules were paid for in falls and are now doctrine. **(a) Spawn from the measured floor**: pelvis height equals the local raycast hit plus 0.76 m—and the ray must be cast with the robot parked elsewhere, or it measures the robot's own head. **(b) The loop time base must match the stepper**: an interactive render-step advances one render frame ($1/60$ s = two physics substeps), not one physics step; running the gait clock on the wrong base halves its speed and the policy falls within half a second. The same bug had the interactive conveyor running at half speed for the project's entire prior history—invisible with a welded robot, fatal with a balancing one.

At zero command the blind policy marches in place and drifts ($\sim$0.3 m/s). A station-keeping loop converts parking drift into small body-frame corrections,

$$c_{xy} = \mathrm{clip}\big(1.2\, R^\top e_{xz},\; \pm 0.15\big), \tag{6}$$

as real velocity-command deployments do.

4 Task and verifier

4.1 Task definition

Parcels arrive continuously in the bin via the recirculating loop. For each parcel the operator must **pick it from the pile and place it label-up on the moving take-away**, where the scan tunnel reads it. The environment defines the task; solving it (scripted or learned) is intentionally out of scope for this release.

4.2 Verifier design

The verifier uses only sensors a real sortation line has.

Photo-eye (throughput). A beam segment across the belt at the scan line, mounted at surface +1.2 cm, tested geometrically against rigid oriented bounding boxes and FEM surface points. At a +6 cm mount, 4.2 cm mailers and 1.8 cm flats passed underneath uncounted; the low mount is a measured fix. A rising edge counts one processed parcel.

Optical scan (correctness). While the beam is blocked, the tunnel camera's rendered image is decoded by a real barcode reader [3] every 15 steps; a decode is attributed to the parcel nearest the tunnel. There is *no metadata path*: an unreadable, occluded, or face-down label simply does not decode.

Honesty monitors. Per-step jump/teleport detection on every parcel; any violation fails the episode outright. The pass criterion is

$$\textbf{PASS} \iff \frac{N_{\text{decoded}}}{N_{\text{processed}}} \geq 0.80 \ \wedge \ \text{zero honesty violations}, \tag{7}$$

where the 0.80 gate mirrors realistic single-read rates measured in-environment (12–$13/13$ manifest decode at tunnel optics; belt-pass scanning yields multiple frames per parcel).

4.3 Verifier validation

The verifier was tested like a subsystem, not trusted. Self-tests (geometry, attribution, edge cases) pass. A live validation run with a deliberately label-down mailer counted $3/3$ parcels, decoded 2, correctly refused the face-down label, and bound the gate ($0.67 < 0.80 \Rightarrow$ FAIL). An *injected teleport cheat* (a parcel warped across the scan line) was caught by the jump monitor and failed the episode. One real defect was found during environment audits: the pass expression omitted its documented no-jam term (a run printed PASS with a parcel resting on the floor); the gate now enforces every documented criterion.

5 Validation results

All results in Table 3 are measured from logged runs preserved in the repository; none are estimates.

Table 3: Measured validation results.

Subsystem	Result
Gravity	$(0, -1, 0) \cdot 9.81$ read back every run
Settle	pile parcels rest on spawn surfaces; residual velocity 0.000–0.006 m/s
Conveyance	rigid 0.398, FEM 0.406 m/s at 0.4 command; in-station 0.30–0.35 m/s
Flow audit (standard, 9 parcels)	PASS repeatedly: 0 teleports/leaks/NaN/speed violations; $6/6$ circulators delivered; pile settled
Flow audit (density, 14 in cell)	PASS $\times 4$ reproducible under the strict gate (incl. no-jam term)
Bin capacity	measured: 17–18 in cell sheds 1–2 parcels gently per run (heap at rim); 14 is the shed-free robot-less operating point; higher density requires an operator draining the bin
Label optics	12–$13/13$ manifest decode at the tunnel camera; texture stretch and sub-physical label sizes are decode-killers (fixed)
Verifier	self-tests pass; live validation $5/5$ checks; injected cheat caught
Locomotion (flat)	20 s upright; walked 3.29 m under a 0.5 m/s command; turned on command; PASS at $200/50$ Hz and $120/60$ Hz
Locomotion (in station)	upright on the reconstructed floor; 120 s interactive driving session with 0 falls; station-keeping parks at ± 3 cm
Collision survey	raycast map: real floor $+6$–14 cm; belt 0.75; guard 0.78; tray floor 0.49; guard-panel trim 1.20 (self-consistent with built geometry)
Performance	$\sim$30 ms/step headless (RTX 3090) with FEM + robot; photoreal interactive sessions

Every failure encountered en route is preserved in the repository logs with its diagnosis—including three initially *wrong* diagnoses (a box "cleat escape" that was actually pile rollout; a "reconstruction blob" render occlusion that was a nondeterministic frame-grab flake; a chirality inference drawn from a top-down camera with a degenerate up-vector)—each retracted and corrected by direct measurement. We consider this failure record part of the environment's documentation: it states precisely which pitfalls the current design guards against.

6 Realism methodology

Three rules governed acceptance of every subsystem.

Measure before claiming. No property is asserted from intention. The audit harness (teleport, leak, NaN and speed detectors; segment events; jam detection; settle checks) must pass, and its gate must enforce every documented criterion.

Render before believing. Multiple defects (the discharge fin, a fallen spawn, guard-panel translucency) were invisible in telemetry and caught only by generating and *looking at* frames. No interactive session is handed to a user without a rendered frame of the same scene state.

Reality sets parameters; the simulation does not negotiate. Real masses, dimensions and label sizes; honest friction pairs; a real balance controller; a real decoder. Where the reference footage's physicality mattered (tote proportions, e-stop placement, bag palette), it was compared against renders side-by-side.

7 Declared approximations and limitations

1. **Poly bags are FEM volumetric pillows**, not thin film around discrete contents. Mass, size, friction and compliance are honest; bag construction is not literal. This is the largest single fidelity gap for grasp learning.
2. **Rigid parcels have uniform-density inertia**; real contents sit off-center and shift.
3. **Arm and hand PD gains are plausible but not system-identified** against hardware (leg control *is* the manufacturer's deployment configuration; hand torque caps approximate real Dex3 limits).
4. **The interactive policy rate is $60\,$Hz versus the native $50\,$Hz**—validated stable at both; the command-to-speed gain runs $\approx 1.5\times$ at $60\,$Hz.
5. **The conveyor return run is abstracted** (tiles wrap underground, collision-gated); the working surface is fully physical, and a real belt's return side never touches a parcel.
6. **Balance-during-manipulation is unexplored**: all manipulation reach data in the repository predates the free-standing robot and must be re-established on the balancing base by policy builders.

8 Intended use

The environment targets: (a) training and evaluating pick-and-orient policies under a sensor-realistic, cheat-resistant success signal; (b) teleoperated demonstration collection (a keyboard interface drives locomotion commands, waist, arms and fingers, with a robot-head camera view); and (c) studying loco-manipulation interference—the balance policy visibly fights arm swings and station contacts, precisely the coupling that fixed-base rigs hide. The verifier's scan-rate gate doubles as a sparse task reward; the photo-eye provides throughput; the honesty monitors disqualify degenerate policies by construction.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[2] Unitree Robotics. unitree_rl_gym: reinforcement-learning locomotion for Unitree robots, with deployable pretrained policies. `https://github.com/unitreerobotics/unitree_rl_gym`, 2024.

[3] ZXing-C++ contributors. zxing-cpp: a C++ port of the ZXing barcode scanning library. `https://github.com/zxing-cpp/zxing-cpp`, 2024.

[4] NVIDIA Corporation. PhysX SDK and Isaac Sim: GPU-accelerated rigid, articulated and deformable body simulation. `https://developer.nvidia.com/isaac-sim`, 2025.

[5] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2022.